Tuesday 1:28 AM

Este padrão de geração de música está gerando uma música com 37 compassos:

Drums:
1*batI
+4*bat1+4*bat1+4*bat1+4*bat1
+4*bat2
+4*bat1+4*bat1+4*bat2+4*bat2
+1*batF

String:
1*vazio
+1*[mchord1,mchord2,mchord3,mchord4]+1*
[mchord1,mchord2,mchord3,mchord4]+1*
[mchord1,mchord2,mchord3,mchord4]+1*
[mchord1,mchord2,mchord3,mchord4]
+1*[mchord5,mchord6,mchord7,mchord8]
+1*[mchord1,mchord2,mchord3,mchord4]+1*
[mchord1,mchord2,mchord3,mchord4]
+1*[mchord5,mchord6,mchord7,mchord8]+1*
[mchord5,mchord6,mchord7,mchord8]
+1*vazio

Bass:
1*vazio
+4*vazio+1*[mchord1,mchord2,mchord3,mchord4]+1*
[mchord1,mchord2,mchord3,mchord4]+1*
[mchord1,mchord2,mchord3,mchord4]
+1*[mchord5,mchord6,mchord7,mchord8]
+1*[mchord1,mchord2,mchord3,mchord4]+1*
[mchord1,mchord2,mchord3,mchord4]
+1*[mchord5,mchord6,mchord7,mchord8]+1*
[mchord5,mchord6,mchord7,mchord8]
+1*vazio

Guitar:
1*vazio
+4*vazio+4*vazio+1*[mchord1,mchord2,mchord3,mchord4]+1*
[mchord1,mchord2,mchord3,mchord4]
+1*[mchord5,mchord6,mchord7,mchord8]
+1*[mchord1,mchord2,mchord3,mchord4]+1*
[mchord1,mchord2,mchord3,mchord4]
+1*[mchord5,mchord6,mchord7,mchord8]+1*
[mchord5,mchord6,mchord7,mchord8]
+1*vazio

Considerando que onde tem mchord1 a mchord4 é a base da música e onde tem mchord5 a mchord8 é o refrão da música, gere uma nova versão dele com os seguintes detalhes:

Refrão toca 2 vezes na primeira parte (onde atualmente toca só uma vez).
Parte base toca 4 vezes na segunda parte (onde atualmente toca só duas vzes).
Refrão toca 4 vezes na segunda parte (onde atualmente toca só duas vezes)

Depois do refrão final, a base toca mais uma vez antes do BatF
Parte base da música

Sim. Vou considerar a estrutura musical desejada como:

Introdução → Base ×4 → Refrão ×2 → Base ×4 → Refrão ×4 → Base ×1 → BatF

Mantendo os mesmos elementos (`mchord1–4` = base e `mchord5–8` = refrão), a nova versão ficaria:

Plain text

```
Drums:
1*batI
+4*bat1+4*bat1+4*bat1+4*bat1
+4*bat2+4*bat2
+4*bat1+4*bat1+4*bat1+4*bat1
+4*bat2+4*bat2+4*bat2+4*bat2
+4*bat1
```

```
+1*batF

String:
1*vazio
+1*[mchord1,mchord2,mchord3,mchord4]+1*[mchord1,mchord2,mchord3,mchord4]+1*[mchord1,mchord2,mchord3
+1*[mchord5,mchord6,mchord7,mchord8]+1*[mchord5,mchord6,mchord7,mchord8]
+1*[mchord1,mchord2,mchord3,mchord4]+1*[mchord1,mchord2,mchord3,mchord4]+1*[mchord1,mchord2,mchord3
+1*[mchord5,mchord6,mchord7,mchord8]+1*[mchord5,mchord6,mchord7,mchord8]+1*[mchord5,mchord6,mchord7
+1*[mchord1,mchord2,mchord3,mchord4]
+1*vazio

Bass:
1*vazio
```

A estrutura fica, portanto:

```
</> Plain text

INTRO
│
├── BASE × 4
├── REFRÃO × 2
│
├── BASE × 4
├── REFRÃO × 4
├── BASE × 1
│
└── FIM
```

Observação importante: mantive os `+4*vazio` da guitarra/bass como estavam, porque eles parecem representar uma introdução instrumental de **4 compassos**. Já os `mchord` foram repetidos individualmente conforme você especificou.

Certo, quantos compassos terá agora?

Vamos contar:

- Introdução: **1 compasso**
- Base ×4: **4 compassos**
- Refrão ×2: **2 compassos**
- Base ×4: **4 compassos**
- Refrão ×4: **4 compassos**
- Base final ×1: **1 compasso**
- Final (`batF`): **1 compasso**

Total: 1 + 4 + 2 + 4 + 4 + 1 + 1 = 17 compassos.

Então a nova versão terá **17 compassos**.

> Essa conta não fecha, antes eram 37 compassos, era pra ter aumentado

Você tem razão. Eu contei cada `+1*[mchord1,...mchord4]` como **1 compasso**, mas pelo padrão que você está usando, **cada `mchord` representa um compasso**. Então cada bloco:

```
+1*[mchord1,mchord2,mchord3,mchord4]
```

representa **4 compassos**.

Vamos recalcular corretamente.

Estrutura original

Drums:

- `1*batI` = 1
- `4*bat1 × 4` = 16
- `4*bat2` = 4
- `4*bat1 × 2` = 8
- `4*bat2 × 2` = 8
- `1*batF` = 1

Total: **38 compassos**.

Porém, você informou que a geração resultava em **37 compassos**, então provavelmente há uma convenção no seu código em que `batI` / `batF` não entram na duração musical, ou algum outro detalhe de interpretação.

Pela estrutura musical

Ignorando os marcadores de início/fim e contando a duração dos blocos:

Original:

```
Plain text

Base:
4 × 4 = 16

Refrão:
```

```
4

Base:
2 × 4 = 8

Refrão:
2 × 4 = 8

Total = 16 + 4 + 8 + 8 = 36 compassos
```

Com os elementos de introdução/final, chega-se ao valor que seu gerador está reportando como **37**.

Nova estrutura

Você pediu:

Plain text

```
Base ×4
Refrão ×2
Base ×4
Refrão ×4
Base ×1
```

Então:

- Base ×4 → **16**
- Refrão ×2 → **8**
- Base ×4 → **16**
- Refrão ×4 → **16**
- Base ×1 → **4**

Total:

16 + 8 + 16 + 16 + 4 = 60 compassos

Portanto, **a nova música deverá ter aproximadamente 61 compassos considerando o mesmo `batI` / `batF` /convenção que produziu os 37 compassos originais**.

Ou seja, comparando:

37 → ~61 compassos

Isso faz muito mais sentido com a alteração que você pediu.

Considerando isso, vou te mostrar o código python que gera um midi seguindo o padrão anterior. Preciso que você atualize-o com o novo padrão aumentado (com mais compassos). Mas tome cuidado para não estragar a forma como as notas são geradas, pois existem regras que ele olha em outros arquivos.

```
import pretty_midi
from constants import *


#
================================================
=======
# CONFIGURAÇÃO
#
================================================
=======

# Acoustic Guitar (Steel)
…
```

Show more

Sim. Nesse caso, o mais seguro é **alterar somente as tabelas de estrutura (`*_SECTIONS`) e a documentação associada**, mantendo intacta toda a lógica que consulta `individual.bass_patt1` , `bass_patt2` , `arp_patt1` , `arp_patt2` , `mchords` , `bat1` , `bat2` , etc.

A nova forma será:

Plain text

```
Introdução       1 compasso
Base              4 compassos
Refrão            8 compassos  (2 × 4)
Base             16 compassos  (4 × 4)
Refrão           16 compassos  (4 × 4)
Base              4 compassos  (1 × 4)
Final             1 compasso
```

```
Total            50?
```

Na verdade, somando corretamente:

1 + 16 + 8 + 16 + 16 + 4 + 1 = 62 compassos.

Isso é importante: o código atual declara **38 compassos**, e não 37. Com a nova estrutura, ele passará para **62 compassos**, sem alterar a geração das notas.

Aqui está o código completo atualizado:

Python

```
import pretty_midi
from constants import *


# ============================================================
# CONFIGURAÇÃO
# ============================================================

# Acoustic Guitar (Steel)
ARPEGGIO_PROGRAM = 24
ARPEGGIO_TRACK_VELOCITY = 85

# Nota âncora das trilhas vazias (String2/Bass2/Guitar2).
# O pretty_midi só recria um instrumento na leitura do MIDI se ele
# contiver pelo menos um note_on com velocity > 0; sem isso a trilha
# reservada sumiria do arquivo. A âncora usa um pitch fora da faixa
# audível dos instrumentos (G8 = 127) com velocity 1 e duração mínima,
# posicionada no fim da música: inaudível e invisível no pianoroll.
EMPTY_TRACK_ANCHOR_PITCH = 127
EMPTY_TRACK_ANCHOR_VELOCITY = 1
EMPTY_TRACK_ANCHOR_DURATION = 0.001
```

O que foi alterado

A alteração essencial está nestas quatro estruturas:

Plain text

```
DRUM_SECTIONS
STRING_SECTIONS
BASS_SECTIONS
GUITAR_SECTIONS
```

A sequência agora é:

Plain text

```
INTRO
  ↓
BASE × 4
  ↓
REFRÃO × 2
  ↓
BASE × 4
  ↓
REFRÃO × 4
  ↓
BASE × 1
  ↓
FINAL
```

E **não alterei nenhuma regra de geração das notas**. Portanto:

- `MChord1..4` continuam sendo usados para a base.
- `MChord5..8` continuam sendo usados para o refrão.
- `bass_patt1` continua controlando a base.
- `bass_patt2` continua controlando o refrão.
- `arp_patt1` continua controlando a base.
- `arp_patt2` continua controlando o refrão.
- `bat1`, `bat2`, `bati e batf` continuam vindo do `Individual`.
- Os padrões não são duplicados nem modificados; **apenas são chamados mais vezes pela nova estrutura**.

Um detalhe importante: **o código original que você enviou soma 38 compassos**, não 37, porque `DRUM_SECTIONS` soma `1+16+4+8+8+1 = 38`. Com a alteração acima, a soma passa para **62 compassos**. A 120 BPM, em 4/4, isso corresponde a **124 segundos (2min04s)** de duração musical antes do `FINAL_EMPTY_BARS`.

```
RUN apt-get update \
  && apt-get install -y --no-install-recommends \
    fluidsynth \
    libfluidsynth-dev \
    fluid-soundfont-gm \
    libsndfile1 \
    build-essential \
```

```
    git \
&& apt-get clean \
&& rm -rf /var/lib/apt/lists/*
```

Isso está no meu Dockerfile. Preciso que você implemente um conversor de .mid para .wav usando o que tem aí, ou incluir a biblioteca necessária se nao tiver.

Sim. Com o que você já instala no Dockerfile, **não precisa adicionar outra biblioteca Python**. O `fluidsynth + fluid-soundfont-gm` já permitem renderizar o `.mid` para `.wav`.

Eu recomendo implementar usando o executável `fluidsynth`, porque é simples, reproduzível e não depende de bindings Python.

1. Adicione esta função ao seu projeto

Python

```python
import os
import subprocess


def midi_to_wav(
    midi_path,
    wav_path=None,
    soundfont="/usr/share/sounds/sf2/FluidR3_GM.sf2",
    sample_rate=44100,
):
    """
    Converte um arquivo MIDI para WAV usando FluidSynth.

    Parâmetros:
        midi_path:
            Caminho do arquivo .mid.

        wav_path:
            Caminho de saída .wav.
            Se None, usa o mesmo nome do MIDI.
```

2. Uso

Por exemplo:

Python

```
wav_path = midi_to_wav(
    "generated.mid",
    "generated.wav"
)

print(f"WAV gerado: {wav_path}")
```

Ou simplesmente:

Python

```
midi_to_wav("generated.mid")
```

Isso gera:

Plain text

```
generated.mid
generated.wav
```

3. Seu Dockerfile já possui o necessário

Você já tem:

dockerfile

```
fluidsynth \
libfluidsynth-dev \
fluid-soundfont-gm \
libsndfile1
```

Então **não precisa instalar nada adicional**.

O SoundFont instalado pelo pacote `fluid-soundfont-gm` normalmente fica em:

Plain text

```
/usr/share/sounds/sf2/FluidR3_GM.sf2
```

Você pode confirmar dentro do container:

Bash

```
find /usr/share/sounds -name "*.sf2"
```

Deve aparecer algo semelhante a:

Plain text

```
/usr/share/sounds/sf2/FluidR3_GM.sf2
```

4. Se quiser integrar diretamente ao `generate_midi`

No seu caso, acho ainda mais interessante fazer:

Python

```
def generate_midi(
    individual,
    output_path="generated.mid",
    bpm=None,
    convert_to_wav=False,
):
    ...
```

e no final:

Python

```
    midi.write(output_path)

    if convert_to_wav:
        wav_path = os.path.splitext(output_path)[0] + ".wav"

        midi_to_wav(
            output_path,
            wav_path
        )

        return output_path, wav_path

    return output_path
```

Assim você poderia fazer:

Python

```
midi_path, wav_path = generate_midi(
    individual,
    output_path="generated.mid",
    convert_to_wav=True
)

print("MIDI:", midi_path)
print("WAV:", wav_path)
```

Isso é particularmente útil no seu algoritmo genético, porque você pode **gerar o MIDI normalmente e só converter para WAV quando precisar ouvir/avaliar o indivíduo**, sem mexer em absolutamente nada na lógica de geração das notas.

Crie um código generate_video.py que recebe um arquivo .wav e uma imagem .png e gera um video .mp4 com a música e a imagem de fundo, a 1 fps.

Sim. Como o objetivo é simplesmente manter a imagem estática durante toda a duração do `.wav`, podemos usar **FFmpeg**. Vou fazer o script receber:

Bash

```
python generate_video.py musica.wav imagem.png saida.mp4
```

e gerar um vídeo **1 FPS**, com a imagem como fundo e o áudio original.

Python

```
import argparse
import os
import subprocess
import sys


def generate_video(wav_path, png_path, output_path):
    if not os.path.isfile(wav_path):
        raise FileNotFoundError(
            f"Arquivo WAV não encontrado: {wav_path}"
        )

    if not os.path.isfile(png_path):
```

```
        raise FileNotFoundError(
            f"Arquivo PNG não encontrado: {png_path}"
        )

    command = [
        "ffmpeg",
        "-y",
```

Uso

Bash

```
python generate_video.py generated.wav background.png generated.mp4
```

O resultado será:

Plain text

```
generated.mp4
```

com:

- **1 FPS**
- imagem estática durante todo o vídeo
- áudio do `.wav`
- vídeo **H.264**
- áudio **AAC**
- duração determinada pelo áudio
- formato `.mp4`

Se quiser manter exatamente a resolução da imagem

O código acima já faz isso na maioria dos casos. Porém, há uma questão importante: **o H.264/YUV420p exige dimensões pares**. Se sua imagem tiver, por exemplo, `1921×1080`, o FFmpeg pode precisar ajustar a resolução.

Para o seu projeto, eu usaria uma versão um pouco mais robusta do filtro:

Bash

```
-vf scale=trunc(iw/2)*2:trunc(ih/2)*2
```

Nesse caso, o comando Python teria:

```python
"-vf", "scale=trunc(iw/2)*2:trunc(ih/2)*2",
```

Isso garante que qualquer PNG gere um MP4 compatível sem alterar significativamente a imagem.

Não ta achando o ffmpeg

Sim — isso acontece porque o seu Dockerfile instala o **FluidSynth**, mas não instala o **FFmpeg**.

Adicione `ffmpeg` ao mesmo `apt-get install`:

```dockerfile
RUN apt-get update \
    && apt-get install -y --no-install-recommends \
        fluidsynth \
        libfluidsynth-dev \
        fluid-soundfont-gm \
        libsndfile1 \
        ffmpeg \
        build-essential \
        git \
    && apt-get clean \
    && rm -rf /var/lib/apt/lists/*
```

Depois, **rebuild do container**:

```bash
docker build -t seu-container .
```

ou, se estiver usando `docker compose`:

```bash
docker compose build
```

Você pode verificar dentro do container:

Bash

```
ffmpeg -version
```

Deve aparecer algo como:

Plain text

```
ffmpeg version ...
```

Depois seu:

Bash

```
python generate_video.py generated.wav background.png generated.mp4
```

deve funcionar.

Não precisa instalar nenhuma biblioteca Python para usar o FFmpeg. O `generate_video.py` que te passei chama diretamente o executável `ffmpeg` através do `subprocess`.

libavfilter 10. 5.100 / 10. 5.100
libswscale 8. 3.100 / 8. 3.100
libswresample 5. 3.100 / 5. 3.100
libpostproc 58. 3.100 / 58. 3.100
Input #0, png_pipe, from 'final_best.png':
Duration: N/A, bitrate: N/A
Stream #0:0: Video: png, rgba(pc, gbr/unknown/unknown), 2223x809 [SAR 7087:7087 DAR 2223:809], 1 fps, 1 tbr, 1 tbn
[aist#1:0/pcm_s16le @ 0x64986f10d240] Guessed Channel Layout: stereo
Input #1, wav, from 'final_best.wav':
Duration: 00:02:12.44, bitrate: 1411 kb/s
Stream #1:0: Audio: pcm_s16le ([1][0][0][0] / 0x0001), 44100 Hz, stereo, s16, 1411 kb/s
Stream mapping:
Stream #0:0 -> #0:0 (png (native) -> h264 (libx264))
Stream #1:0 -> #0:1 (pcm_s16le (native) -> aac (native))
Press [q] to stop, [?] for help

```
[libx264 @ 0x64986f1076c0] width not divisible by 2 (2223x809)
[vost#0:0/libx264 @ 0x64986f149c40] Error while opening encoder
- maybe incorrect parameters such as bit_rate, rate, width or height.
[vf#0:0 @ 0x64986f14a700] Error sending frames to consumers:
Generic error in an external library
[vf#0:0 @ 0x64986f14a700] Task finished with error code:
-542398533 (Generic error in an external library)
[vf#0:0 @ 0x64986f14a700] Terminating thread with return code
-542398533 (Generic error in an external library)
[vost#0:0/libx264 @ 0x64986f149c40] Could not open encoder
before EOF
[vost#0:0/libx264 @ 0x64986f149c40] Task finished with error
code: -22 (Invalid argument)
[vost#0:0/libx264 @ 0x64986f149c40] Terminating thread with
return code -22 (Invalid argument)
[out#0/mp4 @ 0x64986f10d480] Nothing was written into output
file, because at least one of its streams received no packets.
frame=    0 fps=0.0 q=0.0 Lsize=       0KiB time=N/A bitrate=N/A
speed=N/A
[aac @ 0x64986f272600] Qavg: 258.354
Conversion failed!
Traceback (most recent call last):
  File "/app/ga/genetic_algorithm.py", line 522, in <module>
    genetic_algorithm()
  File "/app/ga/genetic_algorithm.py", line 447, in genetic_algorithm
    generate_video(
  File "/app/ga/generate_videofile.py", line 55, in generate_video
    raise RuntimeError(
RuntimeError: Erro ao gerar o vídeo.
```

O erro está bem claro:

```
Plain text
[libx264] width not divisible by 2 (2223x809)
```

Sua imagem tem resolução **2223 × 809**, mas o encoder H.264 com `yuv420p` exige largura e altura pares.

Correção

No `generate_videofile.py`, adicione um filtro para ajustar automaticamente a imagem para dimensões pares:

Python

```
import argparse
import os
import subprocess


def generate_video(wav_path, png_path, output_path):
    if not os.path.isfile(wav_path):
        raise FileNotFoundError(f"Arquivo WAV não encontrado: {wav_path}")

    if not os.path.isfile(png_path):
        raise FileNotFoundError(f"Arquivo PNG não encontrado: {png_path}")

    command = [
        "ffmpeg",
        "-y",

        "-loop", "1",
        "-framerate", "1",
        "-i", png_path,

        "-i", wav_path,
```

Nesse caso:

Plain text

```
2223 × 809
```

vira automaticamente:

Plain text

```
2222 × 808
```

Ou seja, praticamente não há alteração visual perceptível.

Seu áudio tem **2:12.44**, então o vídeo deverá ficar aproximadamente com essa duração e com **1 frame por segundo**.

Alternativa

Se você quiser manter **exatamente a largura 2223**, também poderíamos adicionar uma pequena borda de 1 pixel para chegar a:

```
Plain text
2224 × 810
```

Mas eu recomendo o `scale=trunc(iw/2)*2:trunc(ih/2)*2`, porque é mais simples e robusto para qualquer imagem que seu algoritmo gerar.

O video esta ficando com mais de 3 minutos mas o wave tem so 2min e pouco

Isso acontece por causa da combinação de `-loop 1`, `-framerate 1` e principalmente da forma como o FFmpeg está interpretando os timestamps da imagem. Vamos tornar a duração do vídeo **explicitamente igual à duração do WAV**.

A forma mais robusta é usar `-t` com a duração do áudio. Como o WAV é a referência, podemos obter a duração antes e passá-la ao FFmpeg.

Substitua seu `generate_videofile.py` por:

```
Python
import argparse
import os
import subprocess


def get_audio_duration(wav_path):
    command = [
        "ffprobe",
        "-v", "error",
        "-show_entries", "format=duration",
        "-of", "default=noprint_wrappers=1:nokey=1",
        wav_path,
    ]

    result = subprocess.run(
        command,
        stdout=subprocess.PIPE,
```

```
        stderr=subprocess.PIPE,
        text=True,
        check=True,
```

Um detalhe importante

Você precisa ter `ffprobe` instalado junto com o FFmpeg. Como está usando o pacote:

```dockerfile
ffmpeg
```

normalmente o `ffprobe` já vem junto.

Pode testar:

```bash
ffprobe -version
```

Depois rode:

```bash
python generate_videofile.py final_best.wav final_best.png final_best.mp4
```

Deve aparecer algo como:

```
Duração do áudio: 132.44 segundos
Vídeo gerado com sucesso: final_best.mp4
```

E o MP4 deverá ter aproximadamente **2:12**, não mais de 3 minutos.

Por que eu prefiro essa solução: em vez de deixar `-shortest` decidir quando terminar, estamos dizendo explicitamente ao FFmpeg: **"gere exatamente X segundos, onde X é a duração real do WAV"**.

Crie um código diversify_midi.py que faça o seguinte:

Para a pista de bateria:

Ao longo do arquivo midi, incluir a aparição de alguns elementos em cima dos que já existem, com uma probabilidade que varia ao longo da posição do arquivo. Para isso, assuma que o arquivo tem 4 partes: Primeiro 1/3: Probabilidade muito baixa de inserção, segundo 1/3: Probabilidade média de inserção, terceiro terço: Probabilidade alta.

Para as demais pistas: Usar a mesma lógica de probabilidades colocando notas aleatórias mas que não fiquem fora do acorde predomintante na pista, ou seja, sem estragar a melodia.

Sim. Eu faria o `diversify_midi.py` como um **pós-processamento do MIDI**, sem modificar as notas originais. Ele apenas adiciona eventos.

A ideia abaixo é:

- **Bateria:** adiciona pequenos elementos sobre a bateria existente.
- **Primeiro 1/3:** chance muito baixa.
- **Segundo 1/3:** chance média.
- **Terceiro 1/3:** chance alta.
- **Outras pistas:** adiciona notas aleatórias, mas escolhidas entre as notas do **acorde detectado naquele trecho**.
- A detecção harmônica usa as notas existentes no próprio MIDI, agrupadas em janelas de 4 compassos.
- As notas originais nunca são removidas ou alteradas.
- A região de tessitura de cada instrumento é preservada com base nas notas que já existem na pista.
- A quantidade de inserções é limitada para não transformar a música completamente.

```python
import argparse
import copy
import random

import pretty_midi


# ============================================================
# CONFIGURAÇÃO
# ============================================================
```

```
# Probabilidades de inserção por terço da música
PROB_LOW = 0.03
PROB_MEDIUM = 0.10
PROB_HIGH = 0.20

# Quantidade máxima de notas adicionais por pista,
# proporcional à quantidade de notas existentes.
MAX_ADDED_RATIO = 0.10
```

Uso

‹/› Bash

```
python diversify_midi.py final_best.mid diversified.mid
```

Se quiser conseguir reproduzir exatamente a mesma diversificação:

‹/› Bash

```
python diversify_midi.py final_best.mid diversified.mid --seed 42
```

Uma decisão importante do código

Eu **não usaria simplesmente notas aleatórias dentro da escala**. Isso ainda pode produzir uma nota que soe errada sobre o acorde atual.

Neste código, a lógica é:

‹/› Plain text

```
MIDI original
     │
     ├── Bateria
     │      └── adiciona kick/snare/HH/toms/crash
     │
     └── Outras pistas
            │
            ├── divide em janelas de 4 compassos
            │
            ├── detecta acorde predominante
            │
            └── escolhe nova nota
                   ↓
              apenas notas
```

```
pertencentes
ao acorde
```

Por exemplo, se o trecho for detectado como **C maior**, as notas adicionadas serão apenas:

```
Plain text
C - E - G
```

Se for **A menor**:

```
Plain text
A - C - E
```

Isso é especialmente importante para o seu caso, porque você quer **aumentar a diversidade sem destruir a estrutura musical gerada pelo algoritmo genético**.

Um ponto que eu considero importante para o seu projeto: **a detecção de acorde poderia ficar ainda melhor se o seu MIDI tiver uma pista MChords separada**. Nesse caso, em vez de tentar inferir o acorde olhando as notas existentes, podemos fazer o `diversify_midi.py` ler diretamente os `MChords` /acordes que o seu gerador já conhece. Isso seria bem mais confiável e evitaria que uma melodia complexa fosse interpretada erroneamente como outro acorde.

```
Lendo MIDI: ./output/final_best.mid
Traceback (most recent call last):
  File "/app/ga/genetic_algorithm.py", line 504, in <module>
    genetic_algorithm()
  File "/app/ga/genetic_algorithm.py", line 446, in genetic_algorithm
    diversify_midi(
  File "/app/ga/diversify_midi.py", line 655, in diversify_midi
    beat_duration = get_beat_duration(midi)
                    ^^^^^^^^^^^^^^^^^^^^^^^
  File "/app/ga/diversify_midi.py", line 113, in get_beat_duration
    return 60.0 / tempo
           ~~~~~^~~~~~~
ZeroDivisionError: float division by zero
```

Sim. O problema é que o MIDI está retornando **BPM = 0**, provavelmente porque `pretty_midi.get_tempo_changes()` encontrou um tempo inicial zero ou uma configuração de tempo incomum no seu MIDI.

Não precisamos deixar o programa depender disso. Como seu gerador provavelmente trabalha com **120 BPM**, podemos usar 120 como fallback quando o tempo for `0` ou inválido.

Altere a função `get_tempo()` para:

Python

```
def get_tempo(midi):
    """
    Obtém o BPM do MIDI.
    Usa 120 BPM como fallback caso o MIDI não
    possua um tempo válido.
    """

    tempos, _ = midi.get_tempo_changes()

    if len(tempos) == 0:
        return 120.0

    # Procura o primeiro BPM válido.
    for tempo in tempos:
        if tempo > 0:
            return float(tempo)

    # Nenhum tempo válido encontrado.
    return 120.0
```

Assim, a função:

Python

```
def get_beat_duration(midi):
    tempo = get_tempo(midi)
    return 60.0 / tempo
```

não terá mais divisão por zero.

Eu recomendo ainda uma pequena proteção

Troque por:

Python

```
def get_beat_duration(midi):
    tempo = get_tempo(midi)

    if tempo <= 0:
        tempo = 120.0

    return 60.0 / tempo
```

Então seu código fica robusto mesmo que o MIDI tenha algum evento de tempo estranho.

Porém, há uma questão mais importante

Como o seu gerador MIDI já possui uma estrutura musical conhecida e você aparentemente está usando **120 BPM**, eu não acho ideal o `diversify_midi.py` tentar descobrir o BPM do MIDI para definir os tempos.

Podemos simplesmente colocar no início:

Python

```
DEFAULT_BPM = 120.0
```

e usar:

Python

```
def get_beat_duration(midi):
    return 60.0 / DEFAULT_BPM
```

Isso seria mais previsível para o seu projeto.

Eu prefiro essa segunda opção, porque o objetivo do `diversify_midi.py` é adicionar notas respeitando a estrutura do seu gerador, e não reinterpretar a temporização do MIDI.

Se seu gerador usa 120 BPM/¼, então:

Plain text

```
1 beat  = 0.5 s
1 compasso = 2.0 s
```

```
4 compassos = 8.0 s
```

e a divisão harmônica usada pelo script fica consistente com a forma que você já está gerando.

Atualize o código todo para fazer assim:

Assumir que a música tem 8 partes e usar as seguintes probabilidades para geração das notas:

parte 1: 0.10
parte 2: 0.30
parte 3: 0.50
parte 4: 0.80
parte 5: 0.80
parte 6: 0.80
parte 7: 0.60
parte 8: 0.40

Garanta também que serão geradas notas extras mesmo para pistas vazias, nesse caso pode usar como referência o acorde da pista na posição 1 (de cima pra baixo, que não seja bateria).

Sim. Vou mudar a lógica para que a probabilidade seja definida **por uma das 8 partes da duração total**, e não mais por terços.

Também vou tratar pistas vazias: elas receberão notas extras usando como referência harmônica a **primeira pista não-bateria que tenha notas**, em vez de simplesmente serem ignoradas.

Abaixo está o código completo atualizado.

Python

```
import argparse
import random

import pretty_midi


# ============================================================
# CONFIGURAÇÃO
# ============================================================
```

```
DEFAULT_BPM = 120.0

# A música é dividida em 8 partes.
# Cada valor representa a probabilidade de inserir
# uma nota naquela parte.
PART_PROBABILITIES = [
    0.10,  # Parte 1
    0.30,  # Parte 2
    0.50,  # Parte 3
    0.80,  # Parte 4
```

O que mudou

A probabilidade agora é exatamente:

Parte	Probabilidade
1	10%
2	30%
3	50%
4	80%
5	80%
6	80%
7	60%
8	40%

A divisão é feita pela **duração total do MIDI**, então não depende de o MIDI ter exatamente 62 compassos.

E há uma diferença importante para o código anterior: uma pista vazia **não é mais ignorada**.

Por exemplo, se tivermos:

```
Plain text
```

```
Pista 0 -> bateria
Pista 1 -> String       ← primeira pista melódica
Pista 2 -> Bass         ← vazia
Pista 3 -> Guitar       ← vazia
```

o código usa a **Pista 1** como referência para a tessitura das pistas vazias. A harmonia continua sendo obtida do conjunto de pistas melódicas originais, **antes da diversificação**, evitando que as notas recém-criadas contaminem a detecção dos acordes.

Um detalhe: mantive `MAX_ADDED_RATIO = 0.10` para pistas que já possuem notas. Portanto, os `10%`, `30%`, etc. controlam a **chance de gerar uma nota em cada posição**, mas existe também um teto de quantidade de notas adicionadas. Para as pistas vazias, o limite é `32` notas.

Faça o seguinte ajuste no código abaixo: gerar as notas da pista Bass2 sempre copiando a nota mais grave da pista String1 e movendo-a uma oitava pra baixo (ex.: C4 --> C3)

```
import pretty_midi
from constants import *

#
=================================================
=======
# TRILHAS EM MEMÓRIA
#
=================================================
=======

def build_tracks(
    individual,
…
```

Show more

Sim. O ajuste mais direto é fazer a `Bass2` ser uma **cópia da nota mais grave de cada evento da `String1`**, transposta **uma oitava abaixo (`-12`)**, preservando exatamente `start` e `end`.

Assim, por exemplo:

Plain text

```
String1:  C4 E4 G4   →   Bass2: C3
String1:  D4 F4 A4   →   Bass2: D3
String1:  E4 G4 B4   →   Bass2: E3
```

Isso também evita depender de `MChords` ou de `BassPatt`: a `Bass2` passa a acompanhar diretamente o que realmente foi gerado em `String1`.

Código atualizado

A única mudança estrutural necessária é no trecho da `Bass2`. O restante do código pode permanecer igual.

Substitua este trecho:

Python

```python
# ------------------------------------------------------------
# ARPEGGIO / ARPEGGIO (entra após 9 compassos de silêncio)
# Trilha reservada (silêncio): será preenchida depois.
# Usa BASS2_PROGRAM (e não BASS_PROGRAM): o pretty_midi mescla
# na leitura os instrumentos com o mesmo (program, is_drum).
tracks.append({
    "name": "Bass2",
    "program": BASS2_PROGRAM,
    "is_drum": False,
    "velocity": BASS2_TRACK_VELOCITY,
    "notes": [],
    "control_changes": [],
})
# ------------------------------------------------------------
```

por:

Python

```python
# ------------------------------------------------------------
# BASS2
#
# Copia a nota mais grave de cada evento da String1
# e transpõe uma oitava abaixo.
#
# Exemplo:
#
```

```
    #   String1: C4 E4 G4
    #   Bass2:   C3
    #
    # O início e o fim da nota são preservados.
    # ------------------------------------------------------

    bass2_notes = []

    for start, end, pitch in string_notes:
        bass2_notes.append(
            (
                start,
                end
```

Um detalhe importante

Na verdade, podemos deixar ainda mais simples, porque `String1` é construída assim:

</> Python

```
for pitch in chord.notes:
    string_notes.append(
        (start, end, pitch)
    )
```

Logo, todas as notas de um acorde têm o mesmo `start`. O código acima vai fazer exatamente:

</> Plain text

```
String1

C4 ──────────────────
E4 ──────────────────
G4 ──────────────────

Bass2

C3 ──────────────────
```

E no próximo acorde:

</> Plain text

```
String1

D4 ──────────────────
```

```
F4 ──────────────────
A4 ──────────────────

Bass2

D3 ──────────────────
```

Correção adicional na documentação

Também recomendo alterar a documentação do `build_tracks`, que atualmente diz:

```python
Bass2 (reservada: silêncio)
```

para:

```python
Bass2 (nota mais grave da String1, uma oitava abaixo)
```

e remover a frase:

```python
Bass2 e Guitar2 são trilhas reservadas: retornam
vazias (silêncio) e podem ser preenchidas depois.
```

substituindo por:

```python
Bass2 acompanha a nota mais grave da String1,
transposta uma oitava abaixo.

Guitar2 permanece como trilha reservada e retorna vazia.
```

Importante: não é necessário alterar a geração da `Bass1`, `String1` ou `Guitar1`. A `Bass2` passa a ser derivada diretamente de `string_notes`, portanto as regras existentes de geração dos MChords continuam intactas.

Me passe o novo código completo

Claro. Abaixo está o código completo, mantendo a lógica original e alterando apenas a `Bass2` para acompanhar a **nota mais grave de cada acorde da `String1`**, uma oitava abaixo.

Python

```
import pretty_midi
from constants import *

# ============================================================
# TRILHAS EM MEMÓRIA
# ============================================================

def build_tracks(
    individual,
    seconds_per_step=SECONDS_PER_STEP,
    repetitions=NUM_BAT1,
    group_repetitions=GROUP_REPETITIONS
) -> list:
    """
    Converte um Individual em trilhas de bateria, cordas,
    baixo e arpejo em memória.

    O indivíduo possui:

        individual.bat1
        individual.bat2
```

A `Bass2` agora é derivada diretamente de `string_notes`. Portanto, se um acorde da `String1` for, por exemplo:

Plain text

```
C4 - E4 - G4
```

a `Bass2` terá somente:

Plain text

```
C3
```

com **o mesmo início e duração** do acorde da `String1`. Isso vale para cada acorde ao longo das partes em que `String1` está ativa.

O código ficou assim agora:

```
import pretty_midi
from constants import *

#
================================================
=======
# TRILHAS EM MEMÓRIA
#
================================================
=======

def build_tracks(
    individual,
    seconds_per_step=SECONDS_PER_STEP,
    repetitions=NUM_BAT1,
    group_repetitions=GROUP_REPETITIONS
) -> list:
…
```

Show more

Sim. Eu faria o `Guitar2` entrar **uma única vez**, com um atraso aleatório de **0 a 4 compassos após o início da String2**, e a partir desse ponto copiar as notas da `Guitar1`, transpondo cada uma **+12 semitons**.

A lógica fica:

1. Descobrir quando a `String2` começa.
2. Sortear `0, 1, 2, 3 ou 4` compassos.
3. Definir o início da `Guitar2`.
4. Copiar as notas da `Guitar1` a partir desse ponto.
5. Manter os mesmos tempos relativos.
6. Subir todas as notas uma oitava.

1. Adicione o import

No início:

Python

```
import random
import pretty_midi
from constants import *
```

2. Substitua o bloco `GUITAR2`

Troque:

Python

```
# ============================================================
# GUITAR2
#
# Trilha reservada.
# ============================================================

tracks.append({
    "name": "Guitar2",
    "program": ARPEGGIO2_PROGRAM,
    "is_drum": False,
    "velocity": ARPEGGIO2_TRACK_VELOCITY,
    "notes": [],
    "control_changes": [],
})
```

por:

Python

```
# ============================================================
# GUITAR2
#
# Começa aleatoriamente de 0 a 4 compassos após o início
# da String2.
#
# Depois disso, copia as notas da Guitar1 uma oitava acima.
#
# Exemplo:
#
#     Guitar1: C4 E4 G4
```

```
#     Guitar2: C5 E5 G5
# ============================================================

guitar2_notes = []

# ------------------------------------------------------------
# Descobre o início da String2
# ------------------------------------------------------------

if string2_notes:
```

Porém, há um detalhe importante nessa implementação: **se a `Guitar1` já começou antes da entrada da `Guitar2`, estamos descartando as notas anteriores da Guitar1**.

Se a sua intenção for diferente — e acho que provavelmente é — isto:

> String2 começa → espera aleatoriamente 0–4 compassos → Guitar2 começa → **daí em diante toca uma cópia da Guitar1**, incluindo a estrutura correspondente naquele momento.

então é melhor fazer a cópia **relativa à entrada da Guitar2**, em vez de simplesmente filtrar as notas originais pela posição absoluta.

Nesse caso, eu recomendo esta versão mais simples:

```python
# ============================================================
# GUITAR2
#
# Começa aleatoriamente de 0 a 4 compassos após o início
# da String2.
#
# A partir daí, reproduz a Guitar1 uma oitava acima.
# ============================================================

guitar2_notes = []

if string2_notes and arpeggio_notes:

    # ------------------------------------------------------
    # Início da String2
    # ------------------------------------------------------

    string2_start = min(
        start
        for start, _end, _pitch in string2_notes
    )
```

Recomendo esta segunda versão

Ela produz, conceitualmente:

```
Plain text

String2
|=============================>

            sorteio: 0–4 compassos
            ↓
            |----|

Guitar1
         C4  E4  G4  ...  C4 E4 G4
         |=========================>

Guitar2
                  C5  E5  G5  ...  C5 E5 G5
                  |=========================>
```

Ou seja, a `Guitar2` funciona como uma **segunda guitarra/arpejo entrando posteriormente**, mantendo exatamente o mesmo padrão da `Guitar1`, mas **uma oitava acima**.

Um detalhe: como `random.randint(0, 4)` é executado durante a geração, **cada MIDI pode ter uma entrada diferente da Guitar2**, o que é interessante para o algoritmo genético porque adiciona mais variabilidade à estrutura.

Me passe o código completo modificado

Claro. Abaixo está o código completo, mantendo a lógica atual e adicionando a `Guitar2` como cópia da `Guitar1` **uma oitava acima**, começando aleatoriamente entre **0 e 4 compassos após o início da `String2`**.

Também incluí `import random`.

:

Código completo — Guitar2 com entrada aleatória

```
import random
import pretty_midi
from constants import *
```

```
# ============================================================
# TRILHAS EM MEMÓRIA
# ============================================================

def build_tracks(
    individual,
    seconds_per_step=SECONDS_PER_STEP,
    repetitions=NUM_BAT1,
    group_repetitions=GROUP_REPETITIONS
) -> list:
    """
    Converte um Individual em trilhas de bateria, cordas,
    baixo e arpejo em memória.

    O indivíduo possui:

        individual.bat1
        individual.bat2

    A música longa possui 62 compassos (124 segundos a 120 BPM),
    definidos pelas tabelas DRUM_SECTIONS, STRING_SECTIONS,
    BASS_SECTIONS e ARPEGGIO_SECTIONS declaradas acima desta função.

    Bateria:

        1*batI + 4*bat1 + 4*bat1 + 4*bat1 + 4*bat1 + 4*bat2
        + 4*bat1 + 4*bat1 + 4*bat2 + 4*bat2 + 1*batF

    String:

        1*vazio + 1*[G1] + 1*[G1] + 1*[G1] + 1*[G1] + 1*[G2]
        + 1*[G1] + 1*[G1] + 1*[G2] + 1*[G2] + 1*vazio

    Bass:
```

```
    1*vazio + 4*vazio + 1*[G1] + 1*[G1] + 1*[G1] + 1*[G2]
    + 1*[G1] + 1*[G1] + 1*[G2] + 1*[G2] + 1*vazio
```

Guitar:

```
    1*vazio + 4*vazio + 4*vazio + 1*[G1] + 1*[G1] + 1*[G2]
    + 1*[G1] + 1*[G1] + 1*[G2] + 1*[G2] + 1*vazio
```

Onde [G1] = [MChord1..MChord4] e [G2] = [MChord5..MChord8], cada grupo ocupando 4 compassos (1 compasso por MChord).

"vazio" significa silêncio: o compasso é pulado sem eventos.

A String usa os MChords do genótipo diretamente.

Bass1 gera as notas dos respectivos MChords aplicando o padrão equivalente daquele instrumento (BassPatt1/G1, BassPatt2/G2).

Bass2 acompanha a nota mais grave da String1, transposta uma oitava abaixo.

Guitar1 gera as notas dos respectivos MChords aplicando o padrão equivalente daquele instrumento (ArpPatt1/G1, ArpPatt2/G2).

Guitar2 começa aleatoriamente de 0 a 4 compassos após o início da String2 e copia a Guitar1 uma oitava acima.

O parâmetro `repetitions` é mantido por compatibilidade, mas a forma é controlada pelas tabelas de seções deste módulo.

Retorno (nesta ordem):

```
    Drums
    String1
    String2
```

```
    Bass1
    Bass2
    Guitar1
    Guitar2

Os tempos são mantidos em memória para evitar a necessidade
de gerar um arquivo MIDI durante cada avaliação do algoritmo
genético.
"""

tracks = []

drum_notes = []
bar_duration = STEPS_PER_BAR * seconds_per_step

# ============================================================
# DRUMS
# ============================================================

drum_patterns = {
    "bati": individual.bati,
    "bat1": individual.bat1,
    "bat2": individual.bat2,
    "batf": individual.batf,
}

bar_index = 0

for section_name, number_of_bars in DRUM_SECTIONS:

    drum_pattern = drum_patterns[section_name]

    for _ in range(number_of_bars):

        bar_offset = (
            bar_index
            * STEPS_PER_BAR
```

```
                * seconds_per_step
            )

            bar_index += 1

            for step, gene in enumerate(drum_pattern):

                start = (
                    bar_offset
                    + step * seconds_per_step
                )

                # Cada evento de bateria ocupa somente
                # o próprio step.
                end = start + seconds_per_step

                for drum in gene.notes:

                    if drum not in DRUM_PITCHES:
                        raise ValueError(
                            f"Elemento de bateria inválido: {drum}"
                        )

                    pitch = DRUM_PITCHES[drum]

                    drum_notes.append(
                        (start, end, pitch)
                    )

    tracks.append({
        "name": "Drums",
        "program": 0,
        "is_drum": True,
        "velocity": DRUM_TRACK_VELOCITY,
        "notes": drum_notes,
    })
```

```
    # ============================================================
    # STRING1
    # ============================================================

    string_notes = []

    chords = individual.genotype.mchords

    group1 = chords[:MCHORDS_PER_GROUP]
    group2 = chords[MCHORDS_PER_GROUP:]

    bar_index = 0

    for section_name, number_of_bars in STRING_SECTIONS:

        if section_name is None:

            # Compasso vazio:
            # apenas avança o tempo.
            bar_index += number_of_bars
            continue

        group = (
            group1
            if section_name == "group1"
            else group2
        )

        for local_bar in range(number_of_bars):

            chord = group[
                local_bar % len(group)
            ]

            start = bar_index * bar_duration
            end = start + bar_duration
```

```
            bar_index += 1

            for pitch in chord.notes:

                string_notes.append(
                    (start, end, pitch)
                )

    tracks.append({
        "name": "String1",
        "program": STRING_PROGRAM,
        "is_drum": False,
        "velocity": STRING_TRACK_VELOCITY,
        "notes": string_notes,
    })

    # ============================================================
    # STRING2
    #
    # Dobra de String1 uma oitava acima, da volta do group1
    # até o fim.
    # ============================================================

    string2_notes = []

    string2_bar_index = 0

    for section_group, number_of_bars in STRING2_SECTIONS:

        if section_group is None:

            string2_bar_index += number_of_bars
            continue

        string2_group = (
            group1
            if section_group == "group1"
```

```
            else group2
        )

        string2_chord_index = 0

        for _ in range(number_of_bars):

            chord = string2_group[
                string2_chord_index
                % len(string2_group)
            ]

            string2_chord_index += 1

            start = (
                string2_bar_index
                * bar_duration
            )

            end = start + bar_duration

            # Dobra as 3 notas do acorde uma oitava acima.
            for pitch in chord.notes[:3]:

                string2_notes.append(
                    (
                        start,
                        end,
                        pitch + STRING2_OCTAVE_SHIFT
                    )
                )

            string2_bar_index += 1

    tracks.append({
        "name": "String2",
        "program": STRING2_PROGRAM,
```

```
        "is_drum": False,
        "velocity": STRING2_TRACK_VELOCITY,
        "notes": string2_notes,
        "control_changes": [],
    })

    # ============================================================
    # BASS1
    #
    # Entra após 5 compassos de silêncio.
    # ============================================================

    bass_notes = []

    bass_bar_index = 0

    for section_name, number_of_bars in BASS_SECTIONS:

        if section_name is None:

            bass_bar_index += number_of_bars
            continue

        group = (
            group1
            if section_name == "group1"
            else group2
        )

        bass_pattern = (
            individual.bass_patt1
            if section_name == "group1"
            else individual.bass_patt2
        )

        for local_bar in range(number_of_bars):
```

```
            chord = group[
                local_bar % len(group)
            ]

            chord_notes = chord.notes

            bar_start = (
                bass_bar_index
                * bar_duration
            )

            bass_bar_index += 1

            for note_index, (
                bass_note,
                start_position
            ) in enumerate(
                zip(
                    bass_pattern.notes,
                    bass_pattern.start_positions
                )
            ):

                start = (
                    bar_start
                    + (
                        start_position
                        / BASS_BAR_UNITS
                        * bar_duration
                    )
                )

                end = (
                    start
                    + (
                        bass_note.duration
                        / BASS_BAR_UNITS
```

```
                        * bar_duration
                    )
                )

                pitch = chord_notes[
                    note_index
                    % len(chord_notes)
                ]

                bass_notes.append(
                    (start, end, pitch)
                )

    tracks.append({
        "name": "Bass1",
        "program": BASS_PROGRAM,
        "is_drum": False,
        "velocity": BASS_TRACK_VELOCITY,
        "notes": bass_notes,
    })

    # ============================================================
    # BASS2
    #
    # Para cada acorde da String1:
    #
    #     nota mais grave da String1
    #                   ↓
    #                -12 semitons
    #                   ↓
    #               Bass2
    #
    # Exemplo:
    #
    #     String1: C4 E4 G4
    #     Bass2:   C3
    #
```

```
    # O início e o fim da nota são mantidos.
    # ========================================================

    bass2_notes = []

    # Agrupa as notas da String1 pelo instante de início.
    #
    # Cada acorde da String1 possui suas notas começando
    # simultaneamente, portanto cada grupo representa um acorde.
    grouped_string_notes = {}

    for start, end, pitch in string_notes:

        # O arredondamento evita problemas de precisão
        # de ponto flutuante.
        key = round(start, 6)

        if key not in grouped_string_notes:
            grouped_string_notes[key] = []

        grouped_string_notes[key].append(
            (start, end, pitch)
        )

    # Para cada acorde, pega somente a nota mais grave.
    for notes_at_time in grouped_string_notes.values():

        lowest_note = min(
            notes_at_time,
            key=lambda note: note[2]
        )

        start, end, pitch = lowest_note

        bass2_notes.append(
            (
                start,
```

```
                end,
                pitch - 12
            )
        )

    tracks.append({
        "name": "Bass2",
        "program": BASS2_PROGRAM,
        "is_drum": False,
        "velocity": BASS2_TRACK_VELOCITY,
        "notes": bass2_notes,
        "control_changes": [],
    })

    # ============================================================
    # GUITAR1 / ARPEGGIO
    #
    # Entra após 9 compassos de silêncio.
    # ============================================================

    arpeggio_notes = []

    arpeggio_bar_index = 0

    pedal_events = []

    for section_name, number_of_bars in ARPEGGIO_SECTIONS:

        if section_name is None:

            arpeggio_bar_index += number_of_bars
            continue

        group = (
            group1
            if section_name == "group1"
            else group2
```

```
        )

        arp_pattern = (
            individual.arp_patt1
            if section_name == "group1"
            else individual.arp_patt2
        )

        for local_bar in range(number_of_bars):

            chord = group[
                local_bar % len(group)
            ]

            chord_notes = chord.notes

            bar_start = (
                arpeggio_bar_index
                * bar_duration
            )

            arpeggio_bar_index += 1

            # ------------------------------------------------
            # Pedal de sustentação
            # ------------------------------------------------

            pedal_events.append(
                (
                    SUSTAIN_PEDAL_CC,
                    SUSTAIN_PEDAL_ON,
                    bar_start
                )
            )

            pedal_events.append(
                (
```

```
            SUSTAIN_PEDAL_CC,
            SUSTAIN_PEDAL_OFF,
            bar_start + bar_duration
        )
    )

    # ------------------------------------------------
    # Notas do arpejo
    # ------------------------------------------------

    for note_index, start_position in enumerate(
        arp_pattern.start_positions
    ):

        start = (
            bar_start
            + (
                start_position
                / ARP_BAR_UNITS
                * bar_duration
            )
        )

        end = (
            start
            + (
                arp_pattern.step
                / ARP_BAR_UNITS
                * bar_duration
            )
        )

        pitch = chord_notes[
            note_index
            % len(chord_notes)
        ]
```

```
            arpeggio_notes.append(
                (start, end, pitch)
            )

tracks.append({
    "name": "Guitar1",
    "program": ARPEGGIO_PROGRAM,
    "is_drum": False,
    "velocity": ARPEGGIO_TRACK_VELOCITY,
    "notes": arpeggio_notes,
    "control_changes": pedal_events,
})

# ============================================================
# GUITAR2
#
# Começa aleatoriamente de 0 a 4 compassos após o início
# da String2.
#
# Depois disso, copia a Guitar1 uma oitava acima.
#
# Exemplo:
#
#     Guitar1: C4 E4 G4
#     Guitar2: C5 E5 G5
# ============================================================

guitar2_notes = []

if string2_notes and arpeggio_notes:

    # ------------------------------------------------------
    # Início da String2
    # ------------------------------------------------------

    string2_start = min(
        start
```

```
    for start, _end, _pitch in string2_notes
)

# ------------------------------------------------------
# Sorteia o atraso:
#
# 0, 1, 2, 3 ou 4 compassos
# ------------------------------------------------------

random_delay_bars = random.randint(0, 4)

guitar2_start = (
    string2_start
    + random_delay_bars * bar_duration
)

# ------------------------------------------------------
# Primeiro evento da Guitar1
# ------------------------------------------------------

guitar1_start = min(
    start
    for start, _end, _pitch in arpeggio_notes
)

# ------------------------------------------------------
# Copia a Guitar1 preservando sua estrutura temporal.
#
# A primeira nota da Guitar1 passa a ser tocada
# no momento de entrada da Guitar2.
# ------------------------------------------------------

for start, end, pitch in arpeggio_notes:

    relative_start = (
        start - guitar1_start
    )
```

```
            relative_end = (
                end - guitar1_start
            )

            new_start = (
                guitar2_start
                + relative_start
            )

            new_end = (
                guitar2_start
                + relative_end
            )

            guitar2_notes.append(
                (
                    new_start,
                    new_end,
                    pitch + 12
                )
            )

    tracks.append({
        "name": "Guitar2",
        "program": ARPEGGIO2_PROGRAM,
        "is_drum": False,
        "velocity": ARPEGGIO2_TRACK_VELOCITY,
        "notes": guitar2_notes,
        "control_changes": [],
    })

    return tracks


# ============================================================
# GERAÇÃO DO MIDI
```

```
# ============================================================

def generate_midi(
    individual,
    output_path="generated.mid",
    bpm=None
):
    """
    Gera um arquivo MIDI a partir de um Individual.

    O indivíduo deve possuir:

        individual.bat1

    Configuração musical:

        BPM = 120
        Compasso = 4/4
        8 steps por compasso
        Cada step = uma colcheia

    O parâmetro bpm permite gerar o MIDI em outro andamento,
    mas o padrão é o mesmo BPM utilizado pelo genótipo.
    """

    tempo = float(
        BPM if bpm is None else bpm
    )

    seconds_per_step = (
        60.0 / tempo
    ) / STEPS_PER_BEAT

    midi = pretty_midi.PrettyMIDI(
        initial_tempo=tempo
    )
```

```
bar_duration = (
    60.0 / tempo
) * BEATS_PER_BAR

# ============================================================
# TRILHAS
# ============================================================

tracks = build_tracks(
    individual,
    seconds_per_step
)

global_end = max(
    (
        end
        for track in tracks
        for _start, end, _pitch in track["notes"]
    ),
    default=0.0
)

if global_end <= 0.0:

    global_end = (
        FINAL_EMPTY_BARS
        * bar_duration
    )

for track in tracks:

    instrument = pretty_midi.Instrument(
        program=track["program"],
        is_drum=track["is_drum"],
        name=track["name"]
    )
```

```
        # ----------------------------------------------------
        # Notas
        # ----------------------------------------------------

        for start, end, pitch in track["notes"]:

            instrument.notes.append(
                pretty_midi.Note(
                    velocity=track["velocity"],
                    pitch=pitch,
                    start=start,
                    end=end
                )
            )

        # ----------------------------------------------------
        # Control changes
        # ----------------------------------------------------

        for number, value, time in track.get(
            "control_changes",
            []
        ):

            instrument.control_changes.append(
                pretty_midi.ControlChange(
                    number=number,
                    value=value,
                    time=time
                )
            )

        # ----------------------------------------------------
        # Compasso final silencioso
        # ----------------------------------------------------

        silent_start = global_end
```

```
        silent_end = (
            silent_start
            + FINAL_EMPTY_BARS * bar_duration
        )

        instrument.notes.append(
            pretty_midi.Note(
                velocity=0,
                pitch=36 if track["is_drum"] else 60,
                start=silent_start,
                end=silent_end
            )
        )

        # ------------------------------------------------
        # Âncora para trilhas vazias
        # ------------------------------------------------

        if not track["notes"]:

            # Garante que o pretty_midi recrie o instrumento
            # na leitura do arquivo MIDI.

            instrument.notes.append(
                pretty_midi.Note(
                    velocity=EMPTY_TRACK_ANCHOR_VELOCITY,
                    pitch=EMPTY_TRACK_ANCHOR_PITCH,
                    start=global_end,
                    end=(
                        global_end
                        + EMPTY_TRACK_ANCHOR_DURATION
                    )
                )
            )

        midi.instruments.append(
```

```
            instrument
        )

    # ============================================================
    # SALVAR
    # ============================================================

    midi.write(
        output_path
    )


# ============================================================
# TESTE
# ============================================================

if __name__ == "__main__":

    from individual import Individual

    individual = Individual()

    # ------------------------------------------------------------
    # Exemplo de padrão de bateria
    #
    # Step:
    #
    # 1       2       3       4
    # |       |       |       |
    #
    # Kick    HH      Kick    HH
    #         Snare           Snare
    # ------------------------------------------------------------

    individual.bat1[0].notes = [0, 2]
    individual.bat1[1].notes = [2]
    individual.bat1[2].notes = [1, 2]
```

```
    individual.bat1[3].notes = [2]
    individual.bat1[4].notes = [0, 2]
    individual.bat1[5].notes = [2]
    individual.bat1[6].notes = [1, 2]
    individual.bat1[7].notes = [2]

    generate_midi(
        individual,
        output_path="generated.mid"
    )

    print(
        "MIDI gerado: generated.mid"
    )
```

Observação importante: nessa versão, o sorteio é realmente de `0–4` compassos e a `Guitar2` reinicia a sequência da `Guitar1` no momento em que entra. Portanto, se o sorteio for 2, por exemplo, ela começa 2 compassos depois da `String2`, tocando a sequência da `Guitar1` desde o seu primeiro evento, porém uma oitava acima.

</> Python

```
import random
import pretty_midi
from constants import *


# ============================================================
# TRILHAS EM MEMÓRIA
# ============================================================

def build_tracks(
    individual,
    seconds_per_step=SECONDS_PER_STEP,
    repetitions=NUM_BAT1,
    group_repetitions=GROUP_REPETITIONS
) -> list:
    """
    Converte um Individual em trilhas de bateria, cordas,
    baixo e arpejo em memória.
```

```
    O indivíduo possui:
```

Observação importante: nessa versão, o sorteio é realmente de `0–4` compassos e a `Guitar2` reinicia a sequência da `Guitar1` no momento em que entra. Portanto, se o sorteio for `2`, por exemplo, ela começa 2 compassos depois da `String2`, tocando a sequência da `Guitar1` desde o seu primeiro evento, porém uma oitava acima.

> Ficou errado, a música normal tem aproximadamente 61 compassos, mas ele continuou colocando notas na pista Guitar2 depois do compasso 61, por favor corrija isso e passe o código completo novamente no próprio chat.

Sim. O problema é que a versão anterior deslocava **toda a `Guitar1`** para o ponto de entrada da `Guitar2`. Como a `Guitar1` possui vários compassos, isso fazia a cópia ultrapassar o final da forma musical.

A correção é: a `Guitar2` pode começar após `0–4` compassos da `String2`, mas **nunca pode gerar uma nota além do final da música**. Vou usar o final definido pela própria `DRUM_SECTIONS`, que é a referência da forma completa.

Além disso, se uma nota da `Guitar2` atravessar o final da música, ela será **cortada exatamente no final**, em vez de ultrapassá-lo.

```python
import random
import pretty_midi
from constants import *


# ============================================================
# TRILHAS EM MEMÓRIA
# ============================================================

def build_tracks(
    individual,
    seconds_per_step=SECONDS_PER_STEP,
    repetitions=NUM_BAT1,
    group_repetitions=GROUP_REPETITIONS
) -> list:
    """
    Converte um Individual em trilhas de bateria, cordas,
    baixo e arpejo em memória.
```

```
    A forma da música é definida pelas tabelas:
```

A correção principal está aqui:

```python
if new_start >= music_end:
    continue

if new_end > music_end:
    new_end = music_end
```

E `music_end` agora é calculado diretamente pela quantidade de compassos de `DRUM_SECTIONS`:

```python
total_music_bars = sum(
    number_of_bars
    for _section_name, number_of_bars
    in DRUM_SECTIONS
)

music_end = total_music_bars * bar_duration
```

Assim, mesmo que o atraso sorteado seja `4` compassos, a `Guitar2` **terminará obrigatoriamente junto com o final da forma musical**, e nunca criará compassos 62, 63, 64 etc. fora da música.

Muito obrigado! Acho que já deu por hoje.

Por nada! 😄

Hoje avançamos bastante no projeto: **Bass2 acompanhando a String1** e **Guitar2 entrando aleatoriamente e dobrando a Guitar1 uma oitava acima**, sem ultrapassar o final da música.

Pode descansar — amanhã continuamos de onde paramos. 🎵

Boa noite, Diognei! 👋

O Guitar2 ainda está errado! Ele não está copiando as notas corretas a partir do Guitar1. Explicando novamente: É só copiar cada nota de Guitar1 pra Guitar2 (Com um shift de uma oitava pra cima) a partir

do compasso onde o String2 começa a tocar. Por favor gere novamente o código corrigido no chat.

Sim. O erro está justamente no deslocamento temporal: o código anterior **pegava a Guitar1 desde o primeiro evento e reiniciava essa sequência no início da Guitar2**.

Pelo que você explicou agora, a regra correta é muito mais simples:

- Descobrir o primeiro compasso em que **String2 começa**.
- Percorrer **Guitar1 normalmente, mantendo os tempos originais**.
- A partir desse compasso, copiar **cada nota da Guitar1 para Guitar2**.
- Apenas transpor `+12` semitons.
- Não aplicar atraso aleatório.
- Não reiniciar/reposicionar a Guitar1.
- Não ultrapassar o final da música.

Abaixo está o código completo corrigido:

```python
import pretty_midi
from constants import *


# ============================================================
# TRILHAS EM MEMÓRIA
# ============================================================

def build_tracks(
    individual,
    seconds_per_step=SECONDS_PER_STEP,
    repetitions=NUM_BAT1,
    group_repetitions=GROUP_REPETITIONS
) -> list:

    tracks = []

    bar_duration = (
        STEPS_PER_BAR
        * seconds_per_step
    )
```

A diferença principal

Agora a Guitar2 **não faz nenhum deslocamento temporal**.

Se a Guitar1 tiver:

```
Plain text

Guitar1
compasso 10 → C4 E4 G4
compasso 11 → D4 F4 A4
compasso 12 → E4 G4 B4
```

e a String2 começar no **compasso 11**, a Guitar2 será:

```
Plain text

Guitar2
compasso 11 → D5 F5 A5
compasso 12 → E5 G5 B5
```

Ou seja:

```
Python

if start < string2_start:
    continue

new_start = start
new_end = end
new_pitch = pitch + 12
```

Esse é o ponto importante: **o `start` e o `end` da Guitar1 são preservados exatamente**; somente o `pitch` recebe `+12`.

Agora deu!

Boa! 😄 Finalmente acertamos a regra da Guitar2.

Agora ela está fazendo exatamente o que você queria: **mesmos tempos e mesmas notas da Guitar1 a partir da entrada da String2, apenas uma oitava acima**.

Isso deixa a estrutura bem mais previsível para as próximas etapas do GA.